

GPU COMPUTING WITH CUDA

Programming with GPU follows vector processors in that all processors do the same task simultaneously. In that sense, they offer substantial speed-up over sequential programs, or can over-perform programs written for distributed memory machines.

FROM:

<http://selkie.macalester.edu/>

<http://selkie-macalester.org/csinparallel/modules/>

https://csinparallel.org/csinparallel/modules/timing_mandelbrot_cuda

11.1 What is GPU?

Graphics Processing Unit (GPU), is an electronic circuit that, through rapid memory manipulation and massive parallel data processing, accelerates the building of images intended for output to a display. Right now, GPUs are used in almost all customer end personal computers, game consoles, professional workstations and even the cell phone you are holding.

Before GPU was introduced, CPU did all the graphic processing tasks. In the early 1990s, computer manufactures began to include GPU into computer systems with the aim of accelerating common graphics routines. After two decades of development, GPUs eventually outpaced CPUs as they actually had more transistors, ran faster and were capable of doing parallel computation more efficiently. GPUs became so complex that they are basically computers in themselves, with their own memory, buses and processors. Therefore, sometimes GPU is like an extra brain (supplemental processors) to the computer system. As GPU harnessed more and more horsepower, GPU manufactures, such as NVIDIA and ATI/AMD, found a way to use GPUs for more general purposes, not just for graphics or videos. This gave birth to CUDA structure and CUDA C Programming Language, NVIDIA's

response on facilitating the development of [General Purpose Graphics Processing Unit](#) (GPGPU).

11.2 What is the difference between GPU and CPU?

The major difference between GPU and CPU is that GPU has a highly parallel structure which makes it more effective than CPU if used on data that can be partitioned and processed in parallel. To be more specific, GPU is highly optimized to perform advanced calculations such as floating-point arithmetic, matrix arithmetic and so on.

The reason behind the difference of computation capability between CPU and GPU is that GPU is specialized for compute-intensive and highly parallel computations, which is exactly what you need to render graphics. The design of GPU is more of data processing than data caching and flow control. If a problem can be processed in parallel, it usually means two things: first, same problem is executed for each element, which requires less sophisticated flow control; second, dataset is massive and problem has high arithmetic intensity, which reduces the need for low latency memory.

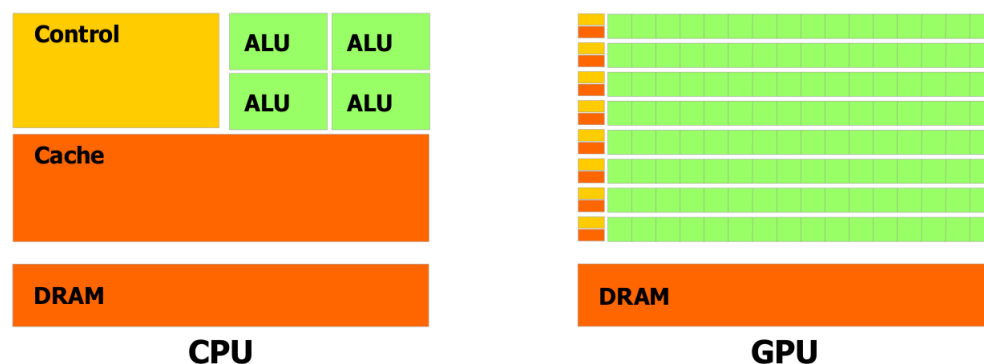


Figure 11.1: This figure is from the [NVIDIA CUDA Programming Guide](#)

The graph above shows the differences between CPU and GPU in their structure. **Cache** is designed for data caching; **Control** is designed for flow control; **ALU** (Arithmetic Logic Unit) is designed for data processing.

In the NVISION 08 Conference organized by the NVIDIA corporation, employers from NVIDIA used a rather interesting yet straight forward example illustrating the difference between CPU and GPU. You can watch the video by clicking the link

below and hope it can give you a better idea about what the difference between GPU and CPU is. [Video](#)

11.3 What is the advantage of using GPU for computation?

Nowadays, most of the scientific researches require massive data processing. What scientist usually do right now is to have all the data being processed on supercomputing clusters. Although most universities have constructed their own parallel computing clusters, researchers still need to compete for time to use the shared resources that not only cost millions to build and maintain, but also consume hundreds of kilowatts of power.

Different from traditional supercomputers that are built with many CPU cores, supercomputers with a GPU structure can achieve same level of performance with less cost and lower power consumption. [Personal Supercomputer](#) (PSC) based on NVIDIA's [Tesla](#) companion processors, was first introduced in 2008. The first generation four-GPU [Tesla](#) personal supercomputer have 4 Teraflops of parallel supercomputing performance, more than enough for most small researches. All it takes is 4 [Tesla](#) C1060 GPUs with 960 cores and two Intel Xeon processors. Moreover, Personal supercomputer is also very energy efficient as it can even run off a 110 volt wall circuit. Although supercomputer with GPUs cannot match the performance of the top ranking supercomputers that cost millions even billions, it is more than enough for researchers to perform daily research related computations in subjects like bioscience, life science, physics and geology.

11.4 What are the important parts inside a GPU?

Although modern GPUs are basically computers themselves, they still serve as a part of a computer system. A modern GPU is connected with the host through a high speed I/O bus slot, usually a PCI-Express slot. Modern GPUs are extremely energy consuming. Some of the GPUs alone consume hundreds watts of power, sometimes higher than all other parts of the computer system combined. Part of the reason that GPUs require such power supply is that they have much complex structure and can perform much sophisticated task than other parts of computer system. Owe to its high capability, GPU needs its own memory, control chipset as well as many processors.

GPUs these days are usually equipped with several gigabytes of on-board memory for user configuration. GPUs designed for daily use like gaming and video render-

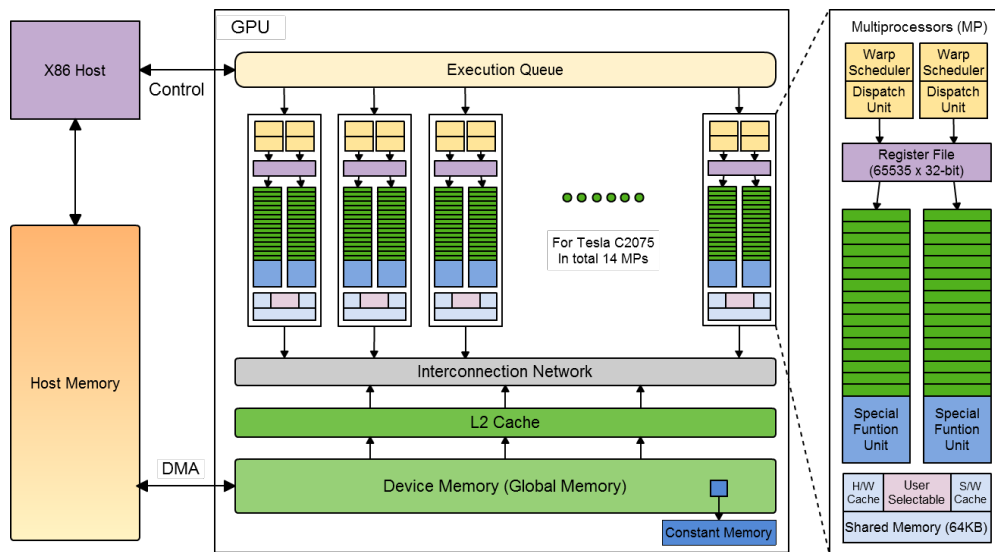


Figure 11.2: This figure is inspired by the figure found in [Understanding the CUDA Data Parallel Threading Model: A Primer](#) written by Michael Wolfe from The Portland Group

ing, such as NVIDIA's [Geforce](#) series GPUs and ATI/AMD's [Radeon](#) series GPUs, have on-board memory capacity ranging from several hundreds megabytes to several gigabytes. Professional GPUs designed for high-definition image processing and General Purpose Computation, such as the [Tesla](#) Companion Processor we are using, usually have memory up to 5 or 6 gigabytes. Data are transferred between the GPU on-board memory and host memory through a method called DMA (Direct Memory Access). One thing worth mentioning is that CUDA C programming language supports direct access of the host memory from GPU end under certain restrictions. As GPU is designed for compute-intensive operations, device memory usually supports high data bandwidth with not deeply cached memory hierarchy.

GPUs from NVIDIA have many processors, called streaming Processors (SP). Each streaming processor is capable of executing a sequential thread. For a GPU with *Fermi* architecture, like the one we are using, every 32 streaming processors is organized in a Streaming Multiprocessor (SM). A GPU can have one or more multiprocessors on board. For example, the [Tesla](#) C2075 GPU card we are using, has 14 multiprocessors built in. Except for 32 streaming processors, each multiprocessor is also equipped with 2 warp scheduler, 2 special function units (4 in some GPUs), a set of 32-bit registers and 64KB of configurable shared memory. A warp scheduler is responsible for threads control; SFU handles transcendentals and double-precision operations. For a GPU with *Kepler* architecture, every 192

streaming processor is organized in a multiprocessor. There are also more warp schedulers and SFUs built in.

Shared memory, or L1 cache, is a small data cache that can be configured through software. Shared memory is also shared among all the streaming processors within one multiprocessor. Compared with on-board memory, shared memory is low-latency (usually register speed) and has high bandwidth. Each multiprocessor has 64KB of shared memory that can be configured by using special commands in host code. Shared memory is distributed to software-managed data cache and hardware data cache. The user can choose to assign either 48KB to software-managed data cache (SW) and 16KB to hardware data cache (HW) or the other way around.

11.5 How does CUDA connect with hardware?

When the host code invokes a kernel grid through a CUDA program, blocks in the grid are distributed to different multiprocessors based on available execution capacity of each multiprocessor. Each multiprocessor is capable of processing one or more blocks throughout the kernel execution. However, each block can only be processed by one multiprocessor.

Fermi architecture supports up to 48 active warps on each multiprocessor. The advantage of having many active warps in a process at the same time is significant reduction of memory latency. Traditionally, memory latency is reduced by adding more cache memory hierarchy into the system. However, by using high degree of multithreading, GPUs can also effectively reduce memory latency. What happens is that when one warp stalls on memory operation, the multiprocessor can select another warp and begin to process that one.

When a block is processed by a multiprocessor, threads in this block are further divided into groups of 32 threads, what NVIDIA calls a warp. Although 32 streaming processors in a block and 32 threads in a warp seems to be a good match for each multiprocessor to process each warp in one clock cycle, the reality is somehow different. As mentioned previously, each multiprocessor has two warp schedulers, which enables it to process two warps simultaneously. After the partition, each warp gets scheduled by a warp scheduler for execution. Each warp scheduler pumps 16 threads (half warp) into a group of 16 streaming processors for execution. Therefore, it would take two clock cycles to process each warp and one multiprocessor can process two warps in two clock cycles. For double-precision operations, each multiprocessor would combine two groups of streaming processors so that they act as a multiprocessor with 16 double-precision streaming processors.

11.6 Is CUDA the only GPU programming language available?

When we are learning CUDA C programming language, it is important for you to know that the C programming language is not the only language that can be bound with CUDA structure. NVIDIA also made other programming languages available such as Fortran, Java and Python as binding languages with CUDA.

Furthermore, NVIDIA is not the only company manufacturing GPU cards, which means CUDA is not the only GPU programming MPI available. When NVIDIA were developing CUDA, AMD/ATI responded with [ATI Stream](#), their GPGPU technology for AMD/ATI Radeon series GPUs. [ATI Stream](#) technology uses [OpenCL](#) as its binding language.

11.7 Introduction to CUDA

When coding with CUDA, we always must distinguish between operations carried out on CPU from the ones done on GPU.

Before we proceed to our first example, please follow the following instructions to set up your working environment. The directory/folder structure needed for these examples is a folder called `GPUProgramming` with two folders inside of it, one called *common* (from a tarball) and one called *examples* (you should make). Here's how to make them:

1. Create a top-level working folder for the code you will examine and run, called `GPUProgramming`.
2. Download the following file: `download common.tar.gz`
3. Extract all the files in `GPUProgramming`, which should create a folder **common**.
4. Create another folder called **examples** inside `GPUProgramming`. The examples folder will contain the CUDA code examples below.
5. Be sure to keep the common folder **outside** of your examples source code folder (this is because the example code includes code from `../common`).

Note: Inside the common folder is the source code that contains helper functions. These include some error handling functions and APIs required for the later graphic programs, such as ray-tracing.

11.8 Acknowledgement

The examples used in this chapter are based on examples in [CUDA BY EXAMPLE: An Introduction to General-Purpose GPU Programming](#), written by Jason Sanders and Edward Kandrot, and published by Addison Wesley.

Copyright 1993-2010 NVIDIA Corporation. All rights reserved.

This copy of code is a derivative based on the original code and designed for educational purposes only. It contains source code provided by [NVIDIA Corporation](#).

11.9 An Example of Vector Addition

We will start our CUDA journey by learning a very simple example, the vector addition example. It basically takes two vectors that have the same dimensions, adds them together and then returns the new vector back.

Vector Addition source file: `VA-GPU-11.cu`

Get this source file and open it in an editor or terminal window so that you can follow along as sections of the code are explained here.

11.9.1 The Device Code

As you may have noticed in your background reading about CUDA programming, CUDA programs execute in two separate places. One is called the host (your CPU), another is called device (your GPU). In our example, the `add()` function executes on the device (our GPU) and the rest of the C program executes on our CPU.

```
__global__ void add( int *a, int *b, int *c ) {  
    int tid = 0;  
    // loop over all the element in the vector  
    while (tid < N){  
        c[tid] = a[tid] + b[tid];  
        tid += 1; // we are using one thread in one block  
    }  
}
```

As shown in the code block above, we need to add a `__global__` qualifier to the function name of the original C code in order to let function `add()` execute on a device.

You might notice that this code is much like standard C code except for the `__global__` qualifier. We are seeing this because this version of vector addition's device code is utilizing only one core of the GPU. We can see this from the line

```
tid += 1; // we are using one thread in one block
```

where we only add 1 to the *tid*. In the later examples, where we will be using more cores of the GPU, you will see differences between CUDA C programming language and Standard C programming language.

11.9.2 The Host Code

Before you proceed

Unlike the device code, the host code is more complicated and requires more explanation. We advise you to download the source file provided at the beginning of this page and have it open in a separate window. We divided the host code into several parts for the purpose of easier explanation. However, looking at the host code as a whole might be helpful, especially for CUDA programming, where host codes are usually highly organized and structured.

```
int main( void ) {

    int *a, *b, *c;
    int *dev_a, *dev_b, *dev_c;

    // allocate memory on the CPU
    a = (int*)malloc( N * sizeof(int) );
    b = (int*)malloc( N * sizeof(int) );
    c = (int*)malloc( N * sizeof(int) );

    // fill arrays 'a' and 'b' on the CPU
    for (int i=0; i<N; i++) {
        a[i] = -i;
        b[i] = i * i;
    }
}
```

As shown in the code block above, similar to standard C programming, we first need to declare pointers. Notice that we declared two sets of pointers, one set is used to store data on host memory, another is used to store data on the device memory.

The Event API

Before we go any further, we need to first learn ways of measuring performance in CUDA runtime. How do we measure performance? That is, how fast can the program run? To be more specific, we will try to time the program.

The tool we use to measure the time GPU spends on a task is the CUDA [Event API](#).

```
// define events in the field
cudaEvent_t    start, stop;
// create two events we need
HANDLE_ERROR( cudaEventCreate( &start ) );
HANDLE_ERROR( cudaEventCreate( &stop ) );
// instruct the runtime to record the event start.
HANDLE_ERROR( cudaEventRecord( start, 0 ) );
```

The first step of using an event is declaring the event. In this example we declared two events, one called start, which will record the start event and another called stop, which will record the stop event. After declaring the events, we can use the command [cudaEventRecord\(\)](#) to record an event. You can think of recording an event as initializing it. You may also notice that we pass this command a second argument (0 in this case). In our example this argument is actually useless. If you are really interested in this, however, you can read more about CUDA streams.

We can see that there is another function `HANDLE_ERROR()` around each of the commands. For the moment, this function does not do anything but returning errors if CUDA commands run into any.

Why would we use Event API?

If you are a C programming language veteran you may ask the question: why don't we use the the timing functions in standard C, such as *clock()* or *timeval* structure, to perform this task? This is a really good question.

The fundamental motivation of using [Event API](#) instead of timing functions in standard C lies on the differences between CPU and GPU computation. To be more specific, GPU is a companion computation device, which means the CPU has to call GPU to do computations every time. However, when the GPU is doing a computation, the CPU does not wait for it to finish its task, instead the CPU will continue to execute the next line of code while GPU is still working on the previous call. This *asynchronous* feature of the GPU computation structure leads to possible inaccuracy when measuring time using standard C timing functions. Therefore, [Event API](#) becomes needed.

The cudaMalloc() Function

```
// allocate memory on the GPU
HANDLE_ERROR( cudaMalloc( (void*)&dev_a, N * sizeof(int) ) );
HANDLE_ERROR( cudaMalloc( (void*)&dev_b, N * sizeof(int) ) );
HANDLE_ERROR( cudaMalloc( (void*)&dev_c, N * sizeof(int) ) );
```

Just like the standard C programming language, you need to allocate memory for variables before you start using them. The command `cudaMalloc()`, similar to `malloc()` command in standard C, tells the CUDA runtime to allocate memory on the device (Memory of GPU), instead of on the host (Memory of CPU). The first argument is a pointer that points to where you want to hold the address of the newly allocated memory.

Because the memory units on the host are separate from those on the GPU, you are not allowed to modify memory allocated on the device (GPU) from the host directly in CUDA C programming language. Instead, you need to use two other methods to access the device memory. You can do it by either using device pointers in the device code, or you can use the `cudaMemcpy()` method.

The way to use pointers in the device code is exactly the same as we did in the host code. In other words, a pointer in CUDA C is exactly the same as in standard C. However, there is one thing you need to pay attention to. Host pointers can only access memory (usually CPU memory) from host code, you cannot access device memory directly. On the other hand, device pointers can only access GPU memory from device code as well.

The cudaMemcpy() Function

```
// copy arrays 'a' and 'b' to the GPU
HANDLE_ERROR( cudaMemcpy( dev_a, a, N * sizeof(int),
                          cudaMemcpyHostToDevice ) );
HANDLE_ERROR( cudaMemcpy( dev_b, b, N * sizeof(int),
                          cudaMemcpyHostToDevice ) );
```

As mentioned in the last section, we can use `cudaMemcpy()` from host code to place data in memory on a device. **This command is the typical way of transferring data between host and device.** This call is similar to the standard C call `memcpy()`, but requires more parameters. The first argument identifies the destination pointer; the second identifies the source pointer. The last parameter to the call is `cudaMemcpyHostToDevice`, telling the runtime that the source pointer is a host pointer and the destination pointer is a device pointer.

The Kernel Invocation

The following line in `main()` is the call for device code to be executed, in this case the function `add()`, which was shown earlier. You may notice that this call is similar to a normal function call but has additional code in it, notably the `<<<`, the `>>>`, and what lies in between them (the triple angle brackets). We will talk about what they represent in later examples. At this point all you need to know is that they are telling the GPU to use only one thread to execute the program.

```
// kernel invocation code
add<<<1,1>>>>( dev_a, dev_b, dev_c );
```

More `cudaMemcpy()` Function

```
// copy array 'c' back from the GPU to the CPU
HANDLE_ERROR( cudaMemcpy( c, dev_c, N * sizeof(int),
                          cudaMemcpyDeviceToHost ) );
```

In the previous section we have seen how CUDA runtime transfers data from Host to Device. This time we will see how to transfer data back to the host. Notice that this time device memory is the source and host memory is the destination. Therefore, we are using the argument `cudaMemcpyDeviceToHost`.

Timing using Event API

```
// get stop time, and display the timing results
HANDLE_ERROR( cudaEventRecord( stop, 0 ) );
HANDLE_ERROR( cudaEventSynchronize( stop ) );
float elapsedTime;
HANDLE_ERROR( cudaEventElapsedTime( &elapsedTime,
                                    start, stop ) );

printf( "Time to generate: %3.1f ms\n", elapsedTime );
```

We have seen how to declare and record an `Event API` in CUDA C, but have not elaborated on how to use such a tool to measure performance. The basic idea is that we first declare an event start and an event stop. Then at the beginning of the program we record event start and at the end of the program we record event stop. The last step is to calculate the elapsed time between two events.

As shown in the code block above, we again use the command `cudaEventRecord()` to instruct the runtime to record the event stop. Then we proceed to the last step, which is to get the elapsed time using the command `cudaEventElapsedTime()`.

However, there is still a problem with timing GPU code in this way. Though

the CUDA C programming language though is derived from standard C, it has many characteristics that are different from standard C. We have mentioned in previous sections that CUDA C is asynchronous. This is an example to jog your memory. Suppose we are running a program to do matrix multiplication, and the host calls the GPU to do the computation. As GPU begins executing our code, the CPU proceeds to the next line of code instead of waiting for the GPU to finish its work. If we want the stop the event to record the correct time, we need to make sure that our event is recorded after the GPU finishes everything prior to the call to `cudaEventRecord()`. To address this problem, CUDA C calls the function `cudaEventSynchronize()` to synchronize the stop event.

The `cudaEventSynchronize()` function is essentially instructing the runtime to create a barrier to block the CPU from executing further instructions until the GPU has reached the stop event.

Another caveat worth mentioning is that CUDA events are implemented directly on the GPU. Therefore they cannot be used for timing device code mixed with host code. In other words, you will get unreliable results if you attempt to use CUDA events to time more than kernel executions and memory copies involving the device.

Note: You should only include kernel execution and memory copies involving the device in between start event and stop event in CUDA. Anything more included could lead to unreliable results.

The `cudaFree()` Function

```
// free memory allocated on the GPU
HANDLE_ERROR( cudaFree( dev_a ) );
HANDLE_ERROR( cudaFree( dev_b ) );
HANDLE_ERROR( cudaFree( dev_c ) );

// destroy events to free memory
HANDLE_ERROR( cudaEventDestroy( start ) );
HANDLE_ERROR( cudaEventDestroy( stop ) );
```

While reading the section about `cudaMalloc()`, it may occur to you that we made a call different from the call `free()` to free memory on the device. To free memory allocated on the device, we need to use command `cudaFree()` instead of `free()`.

To finish up the code, we need to free memory allocated on the CPU as well.

```

// free memory allocated on the CPU
free( a );
free( b );
free( c );

```

The following code is useful to verify whether the GPU has done the task correctly or not. This time we are using CPU to verify GPU's work. We can do this in this problem due to a small data size and simple computation.

```

bool success = true;
for (int i=0; i<N; i++) {
    if ((a[i] + b[i]) != c[i]) {
        printf( "Error:  %d + %d != %d\n", a[i], b[i], c[i] );
        success = false;
    }
}
if (success) printf( "We did it!\n" );

```

11.9.3 Compiling the code and executing this example

From the *examples* folder where your file *VA-GPU-11.cu* is located, you compile the code using the **nvcc** compiler. On unix machines we do this on the command line like this:

```
nvcc -o VA-GPU-11 VA-GPU-11.cu
```

Run the executable called *VA-GPU-11*. On unix machines we do this on the command line like this:

```
./VA-GPU-11
```

This example is only using one thread on the GPU card, so it is not yet a parallel programming example. Furthermore, you would never go to the trouble of moving data just to use one of the many cores available on the GPU card! Continue on to see how we take advantage of those cores.

Note: Before moving on, execute this code a few times and record how much time it takes on your CUDA-enabled GOU.

11.10 Vector Addition with Blocks

We have learned some basic concepts in CUDA C in our last example. Starting from this next example, we will begin to learn how to write CUDA language that will explore the potential of our GPU card.

Download this Vector Addition with Blocks source file: `VA-GPU-N1.cu` into the examples directory where you placed the previous example code file.

11.10.1 Block

Recall that in the previous example, we used the code

```
add<<<1,1>>>( dev_a, dev_b, dev_c );
```

to run device ‘kernel’ code and we left those two numbers in the triple angle brackets unexplained. Well, the first number tells the kernel how many parallel blocks we would like to use to execute the function. For example, if we launch the kernel function with `<<<16,1>>>`, we are essentially creating 16 copies of the kernel function and running them in parallel. CUDA developers call each of these parallel invocations a *block*.

Blocks are organized into a one-dimensional, two-dimensional, or three-dimensional grid. Why do we need a two-dimensional or even a three-dimensional grid? Why can’t we just stick with the one-dimensional grid? Well, it turns out that for problems with two or more dimensional domains, such as matrix multiplication or image processing (don’t forget the reason GPU been exist is to process images faster), it is often convenient and more efficient to use two or more dimensional indexing. Right now, nVidia GPUs that support CUDA structure can assign up to 65536 blocks in each dimension of the grid, that is in total $65536 \times 65536 \times 65536$ blocks in a grid.

Note: Some books imply that the grid in CUDA has at most two-dimensions. On the other hand, some books (including the official CUDA Programming Guide provided by NVIDIA) suggest that the grid can be three-dimensional. It turns out that older GPU units are not powerful enough to run grids in three-dimensions. Therefore older books might argue that CUDA has only a two-dimensional grid. However, as GPUs get more and more powerful, NVIDIA enable newer GPUs to utilize three-dimensional grids.

To see whether your device supports three-dimensional grids or not, please run the following source code and see the Compute Capability entry in the output. If

it is 2.x or 3.x, then your device supports three-dimensional grids. If it is 1.x or less, you can only use two-dimensional grids. [download enum_gpu.cu](#)

11.10.2 The Device Code

```
__global__ void add( int *a, int *b, int *c ) {  
  
    // keep track of the index  
    int tid = blockIdx.x;  
  
    while (tid < N) {  
        c[tid] = a[tid] + b[tid];  
        tid += numBlock; // shift by the total number of blocks  
    }  
}
```

This is the complete device code.

We have mentioned that there are one, two and three-dimensional grids. To index different blocks in a grid, we use the built-in variables CUDA runtime defines for us: `blockIdx`. The variable `blockIdx` is a three-component vector, so that threads can be identified using a one-dimensional, two-dimensional or three-dimensional index. To access different components in this vector, we use `blockIdx.x`, `blockIdx.y` and `blockIdx.z`.

```
// keep track of the index  
int tid = blockIdx.x;
```

Since we have multiple blocks performing the same task, we need to keep track of these blocks so that the kernel can pass the right data to them and bring the right data back. Since we have only 1 thread in each block, we can simply use `blockIdx` to track the index.

```
tid += numBlock; // shift by the total number of blocks
```

Although we have multiple blocks (1 thread per block) working simultaneously after one block finishes one computation, this does not necessarily mean a block will only perform one time of computation. Normally, we could have a problem size that is larger than the number of blocks we have. Therefore, we need each block to perform more than one time of computation. We do this by adding a stride to the `tid` after the while loop finishes one round. In this example, we want `tid` to shift to the next data point by the total number of blocks.

11.10.3 The Host Code

```
add<<<numBlock,numThread>>>( dev_a, dev_b, dev_c );
```

Except the kernel invocation part of the host code, everything else is the same. However, as we are calling **numBlock** and **numThread** in the code, we need to define them at the very beginning of the source code file.

```
#define numThread 1 // in this example we keep one thread in one block  
#define numBlock 128 // in this example we use multiple blocks
```

11.11 Vector Addition with Blocks and Threads

Vector Addition with Blocks and Threads source file: VA-GPU-NN.cu

11.11.1 Threads

In the last example we learned how to launch multiple blocks in CUDA C programs. This time, we will see how to split parallel blocks. CUDA runtime allows us to split blocks into threads. Recall that in the previous example we use the code

```
add<<<numBlock,numThread>>>( dev_a, dev_b, dev_c );
```

to call for device kernels where numBlock is 128 and numThread remain as 1. The second number represents how many threads we want in each block.

Here comes the question, why do we need two sets of parallel organization system? Why do we need not only blocks in grid, but also threads in blocks? Are there any advantages in one over the other? Well, there are advantages that we will cover in later examples, so for now, please bear with us.

Just like blocks are organized in up to three-dimensional grids, threads can also be organized in one, two or three-dimensional blocks. Just like there is a limit on the number of blocks in a grid, there is also a limit on the number of threads in a block. Right now, for most of the high-end nVidia GPUs, this limit is 1024. Be really careful here, 1024 is the total number of threads in a block, **not** the limit **per dimension** like in the grid. Most of the nVidia GPUs that are two or three year old, the limit might be 512. You can query the maxThreadsPerBlock field of the device properties structure to find out which number you have.

11.11.2 The Device Code

```
__global__ void add( int *a, int *b, int *c ) {

    // keep track of the index
    int tid = threadIdx.x + blockIdx.x * numBlock;

    while (tid < N) {
        c[tid] = a[tid] + b[tid];
        tid += numThread * numBlock; // shift by the total number of thread in a grid
    }
}
```

This is the complete device code.

Just like we use CUDA built-in variables to index blocks in a grid, we use variable `threadIdx` to index threads in a block. `threadIdx` is also a three-component vector and you can access each of its element using `threadIdx.x`, `threadIdx.y` and `threadIdx.z`.

```
    // keep track of the index
    int tid = threadIdx.x + blockIdx.x * numBlock;
```

The thread handles the data at its thread id. Recall that earlier we were using `tid = blockIdx.x` only. Now, as we are using multiple threads per block, we have to keep track of not only `blockId`, but also the `threadId` as well.

```
        tid += numThread * numBlock; // shift by the total number of thread in a grid
```

Since we have multiple threads in multiple blocks working simultaneously, after one thread in one block finishes one computation, we want it to shift to the next data point by the total number of threads in the system. In this example, total number of threads is the number of blocks times threads per block.

11.11.3 The Host Code

```
add<<<numBlock,numThread>>>>( dev_a, dev_b, dev_c );
```

Except the kernel invocation part of the host code, everything else is the same. However, as we are calling **numBlock** and **numThread** in the code, we need to define them at the very beginning of the source code file.

```
#define numThread 128 // in this example we use multiple threads
#define numBlock 128 // in this example keep on using multiple blocks
```